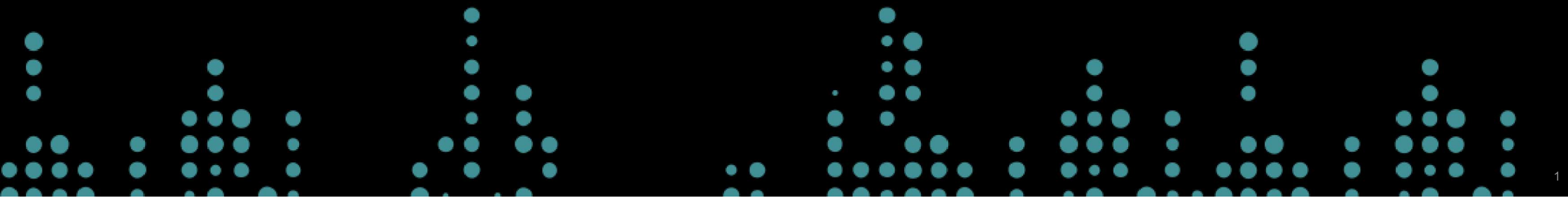


TRANSFORMACIÓN DE DATOS

UNIDAD 5 - Parte 2

Fundamentos de Ciencia de Datos 2026



DICOTOMIZACIÓN DE VARIABLES

La **dicotomización** de una variable cuantitativa consiste en **dividir** los valores observados **en dos grupos** que representan dos categorías exhaustivas y mutuamente excluyentes, a menudo en base a un **punto de corte**.

Esta técnica se utiliza a menudo cuando se desea convertir una variable continua en una variable categórica binaria para su análisis.

DICOTOMIZACIÓN DE VARIABLES

Para ejemplificar, supongamos que tenemos un dataset con información clínica básica de una serie de pacientes:

```
import pandas as pd

# Datos
df = pd.DataFrame({'Paciente': ['P01', 'P02', 'P03', 'P04', 'P05', 'P06', 'P07',
                                'Edad': [45, 62, 38, 71, 55, 29, 48, 66, 33, 57],
                                'IMC': [22.1, 31.4, 27.8, 18.5, 35.2, 24.0, 29.6, 40.1, 23.3, 26.7],
                                'Glucosa_ayunas': [95, 140, 118, 88, 162, 102, 130, 75, 99, 145],
                                'Hipertension': [0, 1, 0, 1, 1, 0, 1, 1, 0, 0]})
```

DICOTOMIZACIÓN DE VARIABLES

```
# Mostramos el dataset
```

```
df
```

	Paciente	Edad	IMC	Glucosa_ayunas	Hipertension
0	P01	45	22.1	95	0
1	P02	62	31.4	140	1
2	P03	38	27.8	118	0
3	P04	71	18.5	88	1
4	P05	55	35.2	162	1
5	P06	29	24.0	102	0
6	P07	48	29.6	130	1
7	P08	66	40.1	75	1
8	P09	33	23.3	99	0

DICOTOMIZACIÓN DE VARIABLES

Podemos ***dicotomizar*** la variable **Glucosa_ayunas** usando el criterio clínico de la **Asociación Americana de Diabetes (ADA)**: una glucosa en ayunas ≥ 126 mg/dL es indicativa de diabetes.

```
df['Diabetes'] = df['Glucosa_ayunas'].apply(  
    lambda x: 'Sí' if x >= 126  
    else 'No')
```

BINNING

```
df[['Paciente', 'Glucosa_ayunas', 'Diabetes']]
```



	Paciente	Glucosa_ayunas	Diabetes
0	P01	95	No
1	P02	140	Sí
2	P03	118	No
3	P04	88	No
4	P05	162	Sí
5	P06	102	No
6	P07	130	Sí
7	P08	75	No
8	P09	99	No
9	P10	145	Sí

BINNING

En este caso, en lugar de dividir los datos de una variable en dos grupos, la división se realiza en grupos múltiples exhaustivos y mutuamente excluyentes llamados **bins** o **buckets** (de ahí lo de **binning**).

BINNING

Por ejemplo, podemos clasificar a los pacientes según su **IMC** en las categorías de la **Organización Mundial de la Salud (OMS)**:

Categoría	Rango de IMC
Bajo peso	$\text{IMC} < 18.5$
Normal	$18.5 \leq \text{IMC} < 25.0$
Sobrepeso	$25.0 \leq \text{IMC} < 30.0$
Obesidad	$\text{IMC} \geq 30.0$

Para ello, una opción rápida es utilizar la función **cut()** de **Pandas**.

BINNING

```
import numpy as np

# Definimos los puntos de corte y los labels
lista_bins = [0, 18.5, 25, 30, np.inf]
etiquetas = ['Bajo peso', 'Normal', 'Sobrepeso', 'Obesidad']

df['Categoria_IMC'] = pd.cut(df['IMC'], bins = lista_bins,
                             labels = etiquetas,
                             right = False) # ¿Qué función cumple este parámetro?
```

BINNING

```
df[['Paciente', 'IMC', 'Categoria_IMC']]
```

	Paciente	IMC	Categoria_IMC
0	P01	22.1	Normal
1	P02	31.4	Obesidad
2	P03	27.8	Sobrepeso
3	P04	18.5	Normal
4	P05	35.2	Obesidad
5	P06	24.0	Normal
6	P07	29.6	Sobrepeso
7	P08	40.1	Obesidad
8	P09	23.3	Normal
9	P10	26.7	Sobrepeso

BINNING



Importante

- Cuando usamos **pd.cut()**, debemos recordar que si queremos definir **n intervalos**, necesitamos un total de **(n + 1) puntos de corte (bins)**.
- El parámetro **right** controla qué extremo del intervalo es cerrado. Por *default* está en **True** (extremo superior cerrado). En este ejemplo usamos **right = False** para que los intervalos sean **[a, b)**, lo que es consistente con los criterios establecidos: un paciente con IMC exactamente igual a 25.0 cae en *Sobrepeso*, no en *Normal*.

BINNING

Si buscamos más flexibilidad (por ejemplo, aplicar una lógica de clasificación más elaborada) podemos usar una función **lambda** de forma explícita:

```
df['Categoria_IMC_2'] = df['IMC'].apply(  
    lambda imc: 'Bajo peso' if imc < 18.5  
    else ('Normal' if imc < 25.0  
    else ('Sobrepeso' if imc < 30.0  
    else 'Obesidad')))
```

BINNING

```
df[['Paciente', 'IMC', 'Categoria_IMC', 'Categoria_IMC_2']]
```

	Paciente	IMC	Categoria_IMC	Categoria_IMC_2
0	P01	22.1	Normal	Normal
1	P02	31.4	Obesidad	Obesidad
2	P03	27.8	Sobrepeso	Sobrepeso
3	P04	18.5	Normal	Normal
4	P05	35.2	Obesidad	Obesidad
5	P06	24.0	Normal	Normal
6	P07	29.6	Sobrepeso	Sobrepeso
7	P08	40.1	Obesidad	Obesidad
8	P09	23.3	Normal	Normal
9	P10	26.7	Sobrepeso	Sobrepeso

CODIFICACIÓN DE VARIABLES CATEGÓRICAS

La mayoría de los algoritmos de aprendizaje automático no pueden manejar variables categóricas a menos que las convirtamos en valores numéricos.

El rendimiento de muchos algoritmos varía en función de cómo se codifican las variables categóricas, por lo que debe analizarse muy bien el método de codificación previo al entrenamiento del modelo.

ONE-HOT ENCODING

En este método, asignamos cada categoría de la variable cualitativa a un **vector que contiene 1 y 0**, con los que se denota la **presencia o ausencia de la característica**, respectivamente. El número de vectores generados dependerá del número de categorías o niveles en la variable de interés.

Para aplicarlo, contamos con la función **get_dummies()** de **Pandas**.

ONE-HOT ENCODING

Agregamos al dataset la variable **Medicamento**, que registra el tratamiento actual de cada paciente:

```
df['Medicamento'] = ['Ninguno', 'Insulina', 'Metformina', 'Ninguno', 'Insulina', 'Ninguno']  
df[['Paciente', 'Medicamento']].head(6)
```

	Paciente	Medicamento
0	P01	Ninguno
1	P02	Insulina
2	P03	Metformina
3	P04	Ninguno
4	P05	Insulina
5	P06	Ninguno

ONE-HOT ENCODING

Aplicamos **get_dummies()** sobre **Medicamento**:

```
df_ohe = pd.get_dummies(data = df, prefix = 'Med', columns = ['Medicamento'])  
df_ohe[['Paciente', 'Med_Insulina', 'Med_Metformina', 'Med_Ninguno']].head()
```

	Paciente	Med_Insulina	Med_Metformina	Med_Ninguno
0	P01	False	False	True
1	P02	True	False	False
2	P03	False	True	False
3	P04	False	False	True
4	P05	True	False	False
5	P06	False	False	True

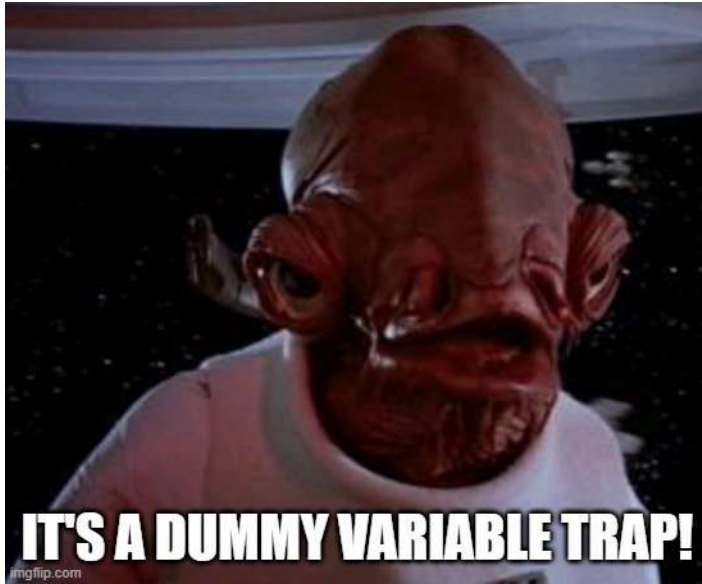
ONE-HOT ENCODING

El parámetro **dtype = 'int'** nos devuelve **0 y 1** en lugar de booleanos.

```
df_ohe2 = pd.get_dummies(data = df, prefix = 'Med', columns = ['Medicamento'])  
df_ohe2[['Paciente', 'Med_Insulina', 'Med_Metformina', 'Med_Ninguno']].head()
```

	Paciente	Med_Insulina	Med_Metformina	Med_Ninguno
0	P01	0	0	1
1	P02	1	0	0
2	P03	0	1	0
3	P04	0	0	1
4	P05	1	0	0
5	P06	0	0	1

DUMMY VARIABLE TRAP



Tener en cuenta que si utilizamos este método de codificación para introducir la información de una variable categórica a un modelo, **debemos considerar (N-1) variables dummy** como predictoras para representar una variable categórica.

⚠ De lo contrario, caemos en la **Dummy Variable Trap**: un problema de **multicolinealidad**.

DUMMY VARIABLE TRAP

En `get_dummies()` esto se resuelve con el parámetro **drop_first = True** (*dropea la dummy para la primera categoría en orden alfabético*):

```
df_ohe = pd.get_dummies(data = df, prefix = 'Med', columns = ['Medicamento'])  
df_ohe[['Paciente', 'Med_Metformina', 'Med_Ninguno']].head(6)
```

	Paciente	Med_Metformina	Med_Ninguno
0	P01	0	1
1	P02	0	0
2	P03	1	0
3	P04	0	1
4	P05	0	0
5	P06	0	1

ORDINAL ENCODING

Se trata de una técnica de preprocesamiento utilizada para convertir datos categóricos de **variables ordinales** en valores numéricos que **preservan su orden inherente**.

De esta manera, presenta dos beneficios:

- Codifica datos categóricos en datos numéricos que los algoritmos pueden entender.
- Retiene la información ordinal entre categorías, que se perdería con otros métodos como *One-Hot Encoding*.

Ejemplos típicos de variables que justifican este método: nivel educativo (*Primaria* → *Secundaria* → *Licenciatura* → *Posgrado*),

ORDINAL ENCODING



Algunas consideraciones importantes

- La codificación ordinal supone un orden secuencial significativo **y unidireccional** entre categorías. No debe usarse para categorías nominales, ni para variables donde “más” no siempre significa lo mismo a lo largo de toda la escala.
- Al asignar enteros consecutivos (0, 1, 2...), el modelo asume que las distancias entre categorías son **iguales**.
- Funciona bien con pocas categorías ordenadas. Con muchas categorías, definir y verificar el orden correcto manualmente se vuelve difícil y propenso a errores.

RESUMEN FINAL

Técnica	Tipo de variable	Ventajas principales	Cuándo evitarla
Dicotomización	Continua	Simplicidad, útil para clasificación binaria	Pérdida de información, arbitrariedad del corte
Binning (cut)	Continua	Agrupación interpretable, reducción de ruido	Pérdida de detalle, depende del corte
One-Hot Encoding	Categórica nominal	No impone orden, funciona bien con modelos lineales	Muchas categorías = alta dimensionalidad
Ordinal Encoding	Categórica ordinal	Preserva orden, simple	No usar con categorías sin orden real